

02285 AI and MAS, F26

Conflict-based search

Curriculum for week 7: Only these slides. *Optional:* The paper *Conflict-based search for optimal multi-agent pathfinding* by Sharon et al. (available on DTU Learn).

Today's subjects:

- Multi-agent pathfinding (MAPF).
- Baseline algorithm for MAPF: Multibody A^* .
- Advanced algorithm for MAPF: Conflict-Based Search (CBS).

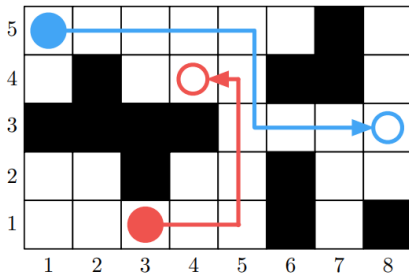
Slides by Mathias Justesen and Thomas Bolander, and with most illustrations from August Møbius' master thesis *Multi-Agent Pathfinding*.

MAPF problem

Has become a very active area of research since 2015. Simpler than the hospital domain:

- No boxes
- All agents have one goal
- But (often) considers general graphs, not only grids.

Today we will look at algorithms for MAPF from the existing literature.



MAPF problem

Definition (MAPF problem)

A **MAPF problem** consists of:

- A graph $G = (V, E)$, where vertices V of the graph are possible locations for the agents, and the edges E are the possible transitions between locations.
- A tuple $(a_1, a_2, \dots, a_k)$ of k agents.
- A start location $v_{i,0}$ for each agent a_i .
- A goal location g_i for each agent a_i .

Definition (Plans and solutions)

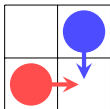
A **plan** for agent a_i is $\pi_i = (v_{i,0}, v_{i,1}, \dots, v_{i,n_i})$ satisfying $v_{i,n_i} = g_i$ and for all $n < n_i$, either $v_{i,n} = v_{i,n+1}$ (wait) or $(v_{i,n}, v_{i,n+1}) \in E$ (move).

A **solution** to a MAPF problem is a tuple $\pi = (\pi_1, \dots, \pi_k)$ of plans, one for each agent, so that there are no *conflicts* between the individual plans (conflicts are defined next...).

Conflicts

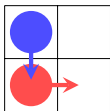
Recall: $v_{i,n}$ is position of agent a_i at time n in plan $\pi = (\pi_1, \dots, \pi_k)$. If $n > |\pi_i|$, we let $v_{i,n} = g_i$ (staying at goal location). Standard conflict types in MAPF (compare Warmup Assignment):

Vertex conflict



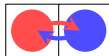
Require for all n and $i \neq j$:
 $v_{i,n} \neq v_{j,n}$

Follow conflict



Require for all n and $i \neq j$:
 $v_{i,n+1} \neq v_{j,n}$

Edge conflict aka swapping conflict



Require for all n, i, j :
 $(v_{i,n}, v_{i,n+1}) \neq (v_{j,n+1}, v_{j,n})$

Why didn't we consider edge conflicts in the hospital domain (Warmup & Theory assignments)?

An edge conflict is also a follow conflict, so disallowing follow conflicts also avoids edge conflicts.

Unless otherwise noted, we here allow follow conflicts (often not a problem for physical robots).

Cost functions

The cost of a solution is most often taken to be one of these:

- **Makespan**: length of longest plan π_i .
- **Sum-of-costs**: sum of lengths of all π_i .
- **Fuel**: like sum-of-costs, except wait actions don't count.

In robotic applications (e.g. at Amazon warehouses), makespan corresponds to total run time, fuel to total energy consumption.

$$\text{makespan}(\pi) = \max_{i=1}^k |\pi_i|$$

$$\text{sum-of-costs}(\pi) = \sum_{i=1}^k |\pi_i|$$

Note that all actions are assumed to have unit costs (no weights on the edges of the graph): The cost of a plan $\pi_i = (v_{i,0}, \dots, v_{i,n_i})$ is $|\pi_i| = n_i$.

Multibody A^* for MAPF

Multibody A^* : A^* graph search algorithm in the space of joint actions. Similar to the Warmup Assignment.

Define for states s and vertices (locations) v, w :

- $v_i(s)$: the location of agent a_i in state s .
- $dist(v, w)$: length of a shortest path from v to w in G (can be pre-computed, e.g., using the Floyd-Warshall algorithm in $O(|V|^3)$ time).

Admissible and consistent heuristics:

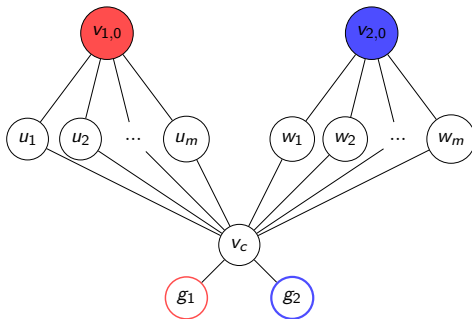
$$h(s) = \sum_{i=1}^k dist(v_i(s), g_i) \quad (\text{for sum-of-costs})$$

$$h(s) = \max_{i=1}^k dist(v_i(s), g_i) \quad (\text{for makespan})$$

Given a consistent heuristic for the chosen cost function, multibody A^* is **complete** (finds a solution if there is one) and **optimal** (finds solution of minimal cost).

Problem (cf. Warmup Assignment): Branching factor grows exponentially with number of agents, and so does the state space size.

Multibody A^* example



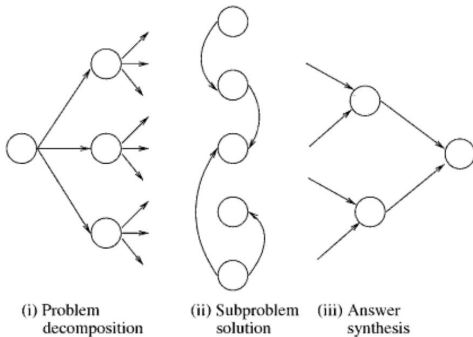
We generate m^2 states at depth 1 of the search tree. They all have the same f -value (makespan: $f = g + h = 1 + 2 = 3$, sum-of-costs: $f = g + h = 2 + (2 + 2) = 6$). For now, consider makespan.

Expanding any state $s = (v_1(s), v_2(s)) = (u_i, w_j)$ at depth 1 gives a child state s' in which one of the agents didn't get any closer to its goal. Hence $f(s') = g(s') + h(s') \geq g(s') + h(s) = g(s) + 1 + h(s) > f(s)$. Thus all m^2 states at depth 1 will be expanded before any state at depth 2. This becomes m^n for n agents. We need something smarter...

Solution: Problem decomposition

To solve the complexity issue of multibody A^* , we can take inspiration in Cooperative Distributed Problem Solving (CDPS) considered earlier in the course:

1. Problem decomposition (one problem per agent)
2. Subproblem solution (A^* for each agent)
3. Solution synthesis (fix conflicts)



Agent-serialised A^*

Agent-serialised A^* : Agents take turn to compute a plan of their own actions, taking the constraints introduced by the plans of the previous agents into account (cf. the Martha project).

But how do agents plan around the constraints introduced by the previous agents (how do they avoid conflicts)?

Definition (Constraint)

(a_i, v, t) is a **constraint** of agent a_i not being allowed to be at vertex $v \in V$ at time t .

To avoid vertex conflicts, when the first agent, a_1 , comes up with its plan $\pi_1 = (v_{1,0}, \dots, v_{1,n_0})$, it leaves a set of constraints for all the later agents to satisfy:

$$\{(a_j, v_{1,n}, n) \mid 2 \leq j \leq k, 0 \leq n \leq n_0\}$$

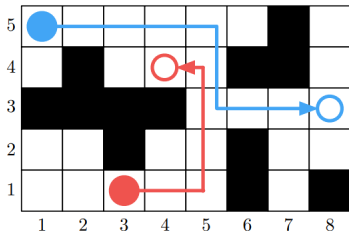
But how does agent a_2 then manage to satisfy these constraints? By doing *space-time A^** , covered next...

Space-time A^*

Space-time A^* : A^* search in the space-time domain, i.e., the state space per agent is $\{(v, t) \mid v \in V, t \in \mathbb{N}\}$, where t denotes the time point.

With space-time A^* we can avoid conflicts by simply deleting the corresponding points in space-time. When agent a_j is planning, it does space-time A^* with all the points (v, t) removed from the state space for which (a_j, v, t) is a constraint.

Example. Suppose red below computed its plan $\pi_r = (v_{r,0}, \dots, v_{r,n_r})$ first (using basic A^*). If blue plans afterwards, and is required not to introduce any vertex conflicts, we can simply delete all the states $(v_{r,n}, n)$ from the space-time domain of blue.

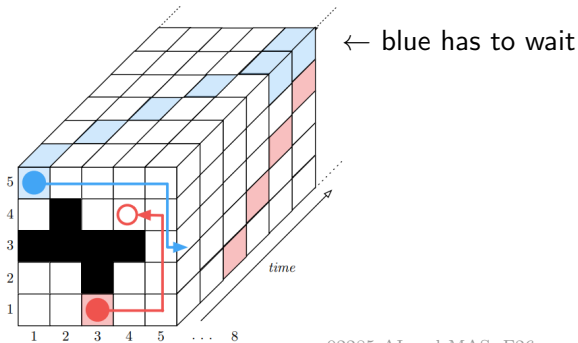


Agent-serialised A^* in more details

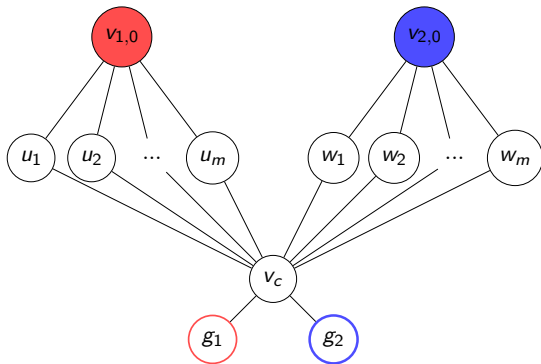
Definition (Agent-serialised A^*)

Agents take turn to compute their plans, using space-time A^* to satisfy the constraints introduced by the previous agents. These constraints are represented as a set of triples (a_i, v, n) where a_i is an agent and (v, n) is a space-time point that a_i is not allowed to use (for vertex conflicts, the constrained space-time points are all those used by a previous agents).

Illustration of agent-serialised A^* with red planning before blue:



Agent-serialised A^* example

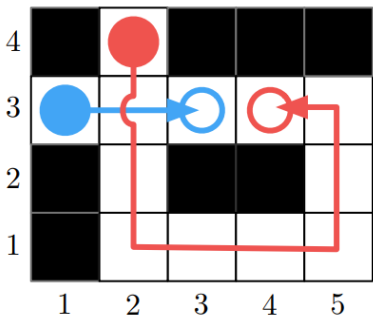


Red can first find an optimal solution using standard A^* (or greedy best-first search, if we don't care about optimality). Afterwards blue plans with the only constraint of staying out of v_c at time 2. We essentially reduce the problem to serialised problem solving (so no exponential growth of branching factor and state space).

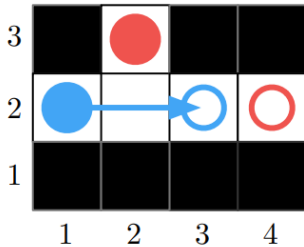
Weaknesses of agent-serialised A^*

Problem: Agent-serialised A^* is neither optimal nor complete.

Suppose blue plans first, and suppose we use makespan as cost function.
Then:



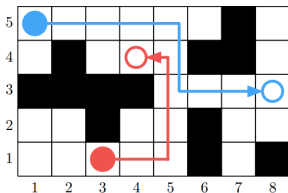
Non-optimality



Incompleteness

Incompleteness comes from MAPF problems not in general satisfying subgoal serialisability (cf. last week's lecture).

From constraints to conflicts



In agent-serialised A^* we let one agent plan before the other and allow the first to induce constraints on the other. This is a kind of *social law* (see later in the course) where some agents take precedence over others.

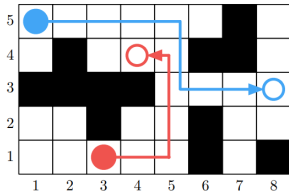
More symmetric or “fair”, we could let each agent plan for itself (using A^*) and then just handle the conflicts that arise (compare to POP).

Definition (Conflict)

(a_i, a_j, v, t) is a **conflict** of agents a_i and a_j both planning to be in location $v \in V$ at time t .

Above we would have the conflict $(b, r, (5, 4), 5)$: blue and red will bump into each other at coordinates $(5, 4)$ at time 5.

Conflict resolution



Recall the conflict $(b, r, (5, 4), 5)$: blue and red will bump into each other at coordinates $(5, 4)$ at time 5.

Then one of them has to back off:

- blue has to satisfy the constraint $(b, (5, 4), 5)$ (stay out of $(5, 4)$ at time 5); *or*
- red has to satisfy the constraint $(r, (5, 4), 5)$.

In other words, each conflict (a_i, a_j, v, n) has two possible resolutions:

1. add constraint (a_i, v, n)
2. add constraint (a_j, v, n) .

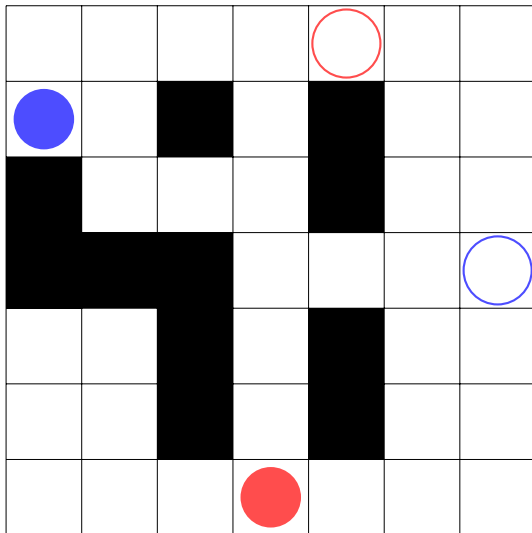
In **conflict-based search (CBS)** one explores both ways of resolving the conflict, and find the best one, preserving optimality.

Conflict-based search (CBS)

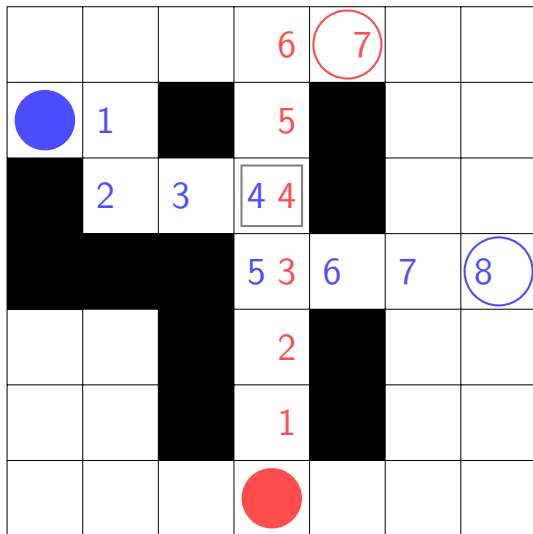
Overall idea behind conflict-based search:

- Use a *high-level* **constraint tree** to keep track of constraints. Each new earliest conflict discovered introduces branching on the different possible conflict resolutions (= different possible added constraints).
- Use a *low-level* space-time A^* search for each agent (taking the constraints on the current branch of the constraint tree into account).

Conflict-based search example



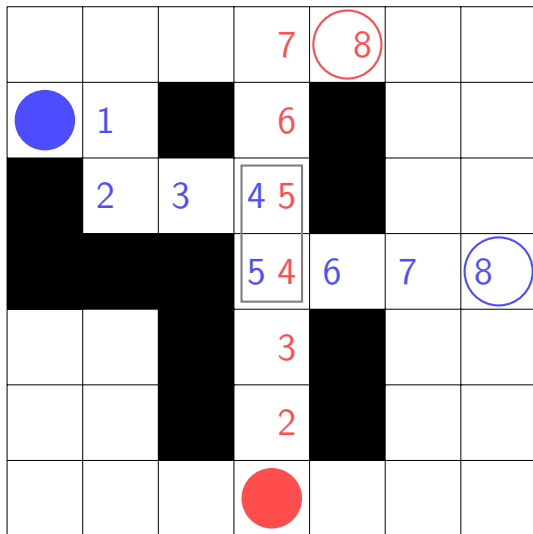
Conflict-based search example



New conflict:
 (•, •, (4, 5), 4)

Sum-of-costs: $8 + 7$

Conflict-based search example



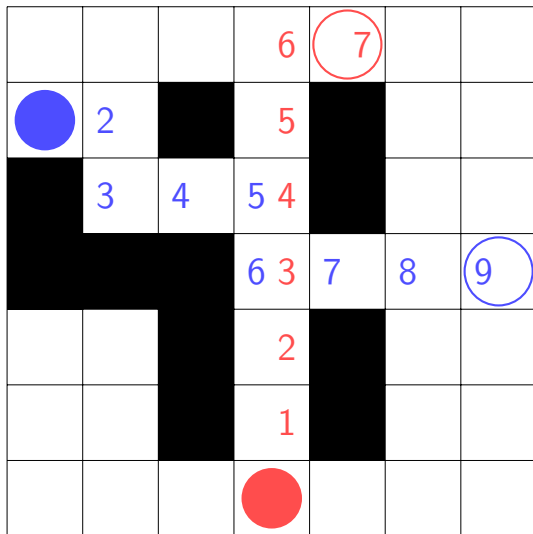
First possible resolution:

$(\bullet, (4, 5), 4)$

Introduces new conflict!
(swapping conflict)

Sum-of-costs: $8 + 8$

Conflict-based search example



Second possible resolution:

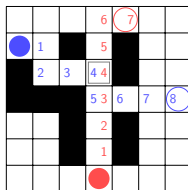
(●, (4, 5), 4)

No conflict!

Compare with plan merging
covered last week

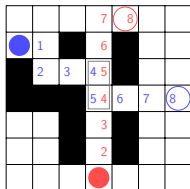
Sum-of-costs: $9 + 7$

Conflict tree: Each node is a relaxed solution satisfying constraints along branch. Do greedy best-first tree search on conflict tree: always expand node of lowest cost.



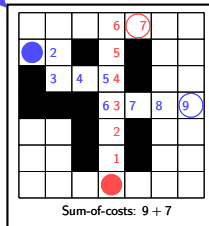
Sum-of-costs: 8 + 7

$(\bullet, (4, 5), 4)$



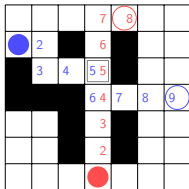
Sum-of-costs: 8 + 8

$(\bullet, (4, 5), 4)$



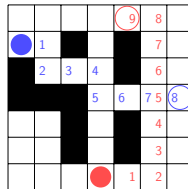
Sum-of-costs: 9 + 7

$(\bullet, (4, 4), 5)$



Sum-of-cost: 9 + 8

$(\bullet, (4, 5), 5)$



Sum-of-cost: 8 + 9

optimal sol. found
cost = 16

Pseudocode for Conflict-Based Search (CBS)

```
def ConflictBasedSearch(problem, agents, cost):
    root.constraints = []
    for a in agents:
        root.solution[a] = AStar(problem[a])
    root.cost = cost(root.solution)
    frontier = PriorityQueue(root, lambda n: n.cost)
    while not frontier.empty():
        node = frontier.get() # get lowest cost
        if IsValid(node.solution):
            return node.solution
        (a1, a2, v, t) = FindFirstConflict(node.solution)
        for a in [a1, a2]:
            m = n.copy()
            m.constraints.append((a, v, t))
            m.solution[a] = SpaceTimeAStar(problem[a],
                                             m.constraints)
            m.cost = cost(m.solution)
            if m.cost < infinity: # solution is found
                frontier.put(m)
```

Optimality and scalability of conflict-based search

Theorem (Completeness and optimality of CBS)

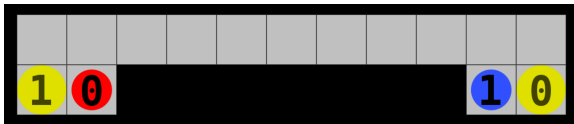
CBS is complete and optimal

Optimality comes from exploring all combinations of constraints.

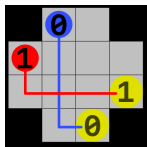
Drawback: high computational cost (cf. Warmup Assignment).

CBS tends to struggle when the subproblems of multiple agents are *strongly coupled* (e.g. two agents passing each other in a corridor).

How many (single-agent) A^* searches will CSB make in a corridor of length n as below? 2 initial searches plus 1 per conflict discovered. One conflict per corridor cell per color = $2n$. So $2n + 2$.



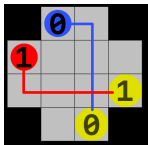
Even worse: In an $n \times n$ open grid there are more than 2^n distinct optimal paths from the lower left corner to the upper right corner! **Why?**
Each of the first n moves can be up or right, giving distinct paths.



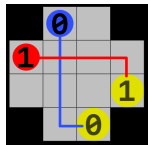
cost: 4 (makespan)

$(\bullet, (2,2), 2)$

$(\bullet, (2,2), 2)$



cost:4



cost:4

$(\bullet, (3,2), 3)$

$\downarrow (\bullet, (3,2), 3)$

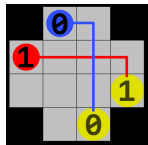
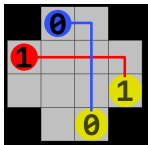
$(\bullet, (2,3), 1)$

$\searrow (\bullet, (2,3), 1)$

solution
blue waits
at (3,3)
cost: 5

cost:4

solution
red waits
at (1,3)
cost: 5



$(\bullet, (3,3), 2)$

$\downarrow (\bullet, (3,3), 2)$

$(\bullet, (3,3), 2)$

$\searrow (\bullet, (3,3), 2)$

solution
cost: 5

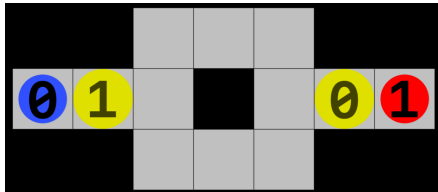
solution
cost: 5

solution
cost: 5

solution
cost: 5

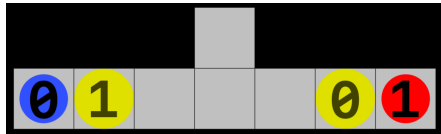
CBS, subgoal serialisability and plan merging

Recall these examples from last week:



Non-serialisable

some optimal solutions mergeable



Non-serialisable

optimal solutions not mergeable

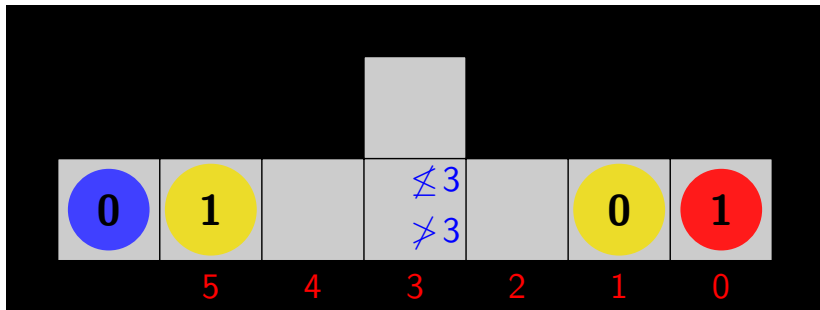
How will CBS handle the left level, assuming there will initially be a conflict in the top middle cell? The conflict generates two possible constraints. Each is solvable by a plan *of the same length* by letting the constrained agent take the other optimal route (going down instead of up). This is a simple merge of two optimal plans.

The right level is more tricky...

CBS example when no plan merge is possible

We here include handling follow conflicts, in order to make the example a bit more interesting/tricky (and to illustrate follow conflicts).

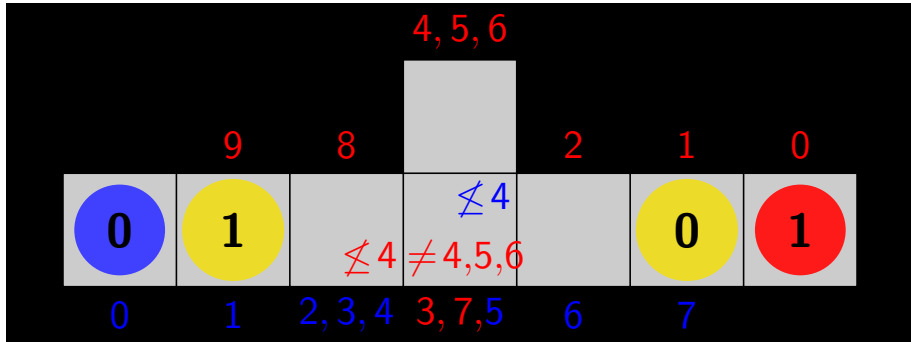
Say that at first we solve each conflicts by constraining blue (letting red stick to its original plan):



No solution found. Symmetric problem if we always constrain red.

Thus we need to interleave constraining blue and red, until a solution is found (which the conflict tree of CBS will of course handle)...

CBS example when no plan merge is possible

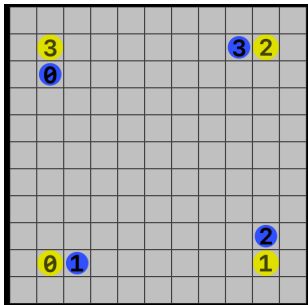


We first constrain blue, as before: $(\bullet, (4, 1), 3)$; swap conflict at times 3–4; constrain red: $(\bullet, (3, 1), 4)$; vertex conflict at time 4; constrain red: $(\bullet, (4, 1), 4)$; follow conflict at times 4–5; constrain blue: $(\bullet, (4, 1), 4)$; vertex conflict at time 5; constrain red: $(\bullet, (4, 1), 5)$; follow conflict at times 5–6; constrain red: $(\bullet, (4, 1), 6)$. Optimal solution found! But many conflicts to handle for problem with state space size $8 \cdot 7 = 56$.

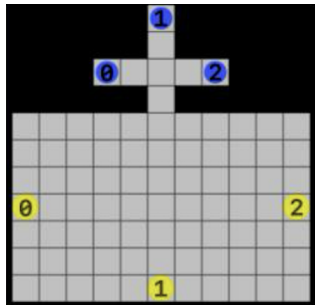
Meta-agent CBS

Many variants and improvements of CBS exists, e.g. **Meta-agent CBS**:

- If there are more than c conflicts between a_i and a_j on a branch, merge them into one *meta-agent* (do multibody A^* for them).
- This idea was also part of the Martha project.
- For $c = 0$, we get an algorithm called **Independence Detection**: Agents plan independently, and if *any* conflict between two agent arises, they are merged into one agent.



Independence-friendly



Not independence-friendly

CBS and the programming project

Don't expect too much from vanilla CBS. It's optimised for optimal solutions, which is generally unrealistic in the programming project (for all but the simplest levels).

Some previous solutions to the programming project inspired by CBS scale poorly, but others scale very well. Just be aware that CBS doesn't do it alone, so if you go for a CBS-inspired solution, make sure to also take inspiration from the other subjects covered in the course.

POP, HTN, the Martha project and CBS are of course all ideas in the right direction: how to make things scale better by divide-and-conquer.

Many non-optimal versions of CBS also exist that scale much better than vanilla CBS.